Chapter One
Introduction to programming

What is programming?

- ***Programming*** is a skill that can be acquired by a computer professional that gives him/her the knowledge of making the computer perform the required operation or task.

Why do we need to learn computer programming?

- ***Computer programming*** is critical if one wants to know how to make the computer perform a task. Most users of a computer only use the available applications on the computer. These applications are produced by computer programmers. Thus if someone is interested to make such kind of applications, he/she needs to learn how to talk to the computer, which is learning computer programming.

What is programming language?

- ***Programming Language:*** is a set different category of written symbols that instruct computer hardware to perform specified operations required by the designer.

What skills do we need to be a programmer?

- For someone to be a programmer, in addition to basic skills in computer, needs to have the following major skills:
  - ***Programming Language Skill***: knowing one or more programming language to talk to the computer and instruct the machine to perform a task.
  - ***Problem Solving Skill***: skills on how to solve real world problem and represent the solution in understandable format.
  - ***Algorithm Development***: skill of coming up with sequence of simple and human understandable set of instructions showing the step of solving the problem. Those set of steps should not be dependent on any programming language or machine.

In every programming Language there are sets of rules that govern the symbols used in a programming language. These set of rules determine how the programmer can make the computer hardware to perform a specific operation. These sets of rules are called ***syntax***.

1.1. Generations of programming language.

- Programming languages are categorized into five generations: (1st, 2nd, 3rd, 4th and 5th generation languages)
- These programming languages can also be categorized into two broad categories: low level and high level languages.
  - Low level languages are machine specific or dependent.
  - High level languages like COBOL, BASIC are machine independent and can run on variety of computers.
- From the five categories of programming languages, first and second generation languages are low level languages and the rest are high level programming languages.
- The higher the level of a language, the easier it is to understand and use by programmers.
- Languages after the fourth generation are referred to as a very high level language.

1.1.1. First Generation (Machine languages, 1940's):

- Difficult to write applications with.
- Dependent on machine languages of the specific computer being used.
- Machine languages allow the programmer to interact *directly with the hardware*, and it can be executed by the computer without the need for a translator.
- Is more powerful in utilizing resources of the computer.
- Gives power to the programmer.
- They execute very quickly and use memory very efficiently.

1.1.2. Second Generation (Assembly languages, early 1950's):

- Uses symbolic names for operations and storage locations.
- A system program called an *assembler* translates a program written in assembly language to machine language.
- Programs written in assembly language *are not portable*. i.e., different computer architectures have their own machine and assembly languages.
- They are *highly used in system software development*.

1.1.3. Third Generation (High level languages, 1950's to 1970's):

- Uses English like instructions and mathematicians were able to define variables with statements such as Z = A + B
- Such languages are much easier to use than assembly language.
- Programs written in high level languages need to be translated into machine language in order to be executed.

- The use of common words (reserved words) within instructions makes them easier to learn.
- All third generation programming languages are procedural languages.
- In procedural languages, the programmer is expected to specify what is required and how to perform it.

1.1.4. Fourth Generation (since late 1970's):

- Have a simple, English like syntax rules; commonly used *to access databases*.
- Forth generation languages are *non-procedural* languages.
- The non-procedural method is easier to write, but you have less control over how each task is actually performed.
- In non-procedural languages the programmer is not required to write traditional programming logic. Programmers concentrate on defining the input and output rather than the program steps required.
    - For example, a command, such as LIST, might display all the records in a file on screen, separating fields with a blank space. In a procedural language, all the logic for inputting each record, testing for end of file and formatting each column on screen has to be explicitly programmed.
- Forth generation languages have a minimum number of syntax rules. This saves time and free professional programmers for more complex tasks.
- Some examples of 4GL are structured query languages (SQL), report generators, application generators and graphics languages.

1.1.5. Fifth Generation (1990's):

- These are used in artificial intelligence (AI) and expert systems; also used for accessing databases.
- 5GLs are "natural" languages whose instruction *closely resembles* human speech. E.g. "get me Jone Brown's sales figure for the 1997 financial year".
- 5GLs require very powerful hardware and software because of the complexity involved in interpreting commands in human language.

1.2. Overview of Computers and Computer Organization.

- Regardless of differences in physical appearance, virtually every computer may be envisioned as being divided into six logical units or sections:
- <u>*Input Unit*</u>: it obtains information from various input devices and places this information at the disposal of the other units so that the information may be processed or stored.
- <u>*Output Unit*</u>: it takes information that has been processed by the computer and places it on various output devices to make the information available for use outside the computer.

- *Memory unit*: it retains information that has been entered through the input unit, so the information may be made immediately available for processing when it is needed. The memory unit also retains processed information until that information can be placed on output devices by the output unit.
- *Central Processing Unit (CPU)*: it is the computer's coordinator and is responsible for *supervising* the operations of the other sections. CPU tells the input unit when information should be read into memory, tells ALU when information from the memory should be used in calculation and tells output unit when to send information from the memory to certain output devices.
- *Arithmetic and Logic unit*: is a part found inside the Central Processing Unit and is responsible for performing calculations such as addition, subtraction, multiplication and division. It also performs comparisons.
- *Secondary Storage device*: Programs or data used by other units normally are placed on secondary storage devices (such as disks) until they are needed, possibly hours, days, months, or even years later.

1.3. The evolution of Operating Systems.

- Early computers were capable of performing only *one job at a time*. This form of computer operation is often called *single-user batch processing*. The computer runs a single program at a time while processing data in groups or batches.
- As computers became more powerful, it became evident that single-user batch processing rarely utilizes the computer's resources.
- Therefore, computer developers thought that many jobs or tasks from different programs or applications could be made to share the resources of the computer to achieve better utilization. This concept is called *multiprogramming*.
- Multiprogramming involving the "simultaneous" operation of many jobs on the computer.
- In the 1960's, many groups, pioneered *timesharing operating systems*.
- Timesharing is a special case of multiprogramming, in which users access the computers through terminals. Here, the computer does not run the users job paralleling, but shares the CPU's time.

1.4. Major Programming Paradigms

- The major land marks in the programming world are the different kinds of features or properties observed in the development of programming languages. Among these the following are worth mentioning: Procedural, Structured and Object Oriented Programming Paradigms.

1.4.1. Procedural Programming.

- Procedural programming is a programming paradigm based upon the concept of *procedure call*. Procedural programming is often a better choice than simple sequential programming in many situations which involve moderate complexity or which require significant ease of maintainability.

- Possible benefits: the ability to re-use the same code (function or procedure) at different places, an easier way to keep track of program flow than a collection of "GO TO" or "JUMP" statements.

1.4.2. Structured Programming.

- Process of writing a program in small, independent parts. This makes it easier to control a program's development and to design and test its individual component parts.
- Structured programs are built up from units called ***modules***, which normally correspond to single procedures or functions.
- Can be seen as a *subset or sub discipline of procedural programming*. It is most famous for removing or reducing reliance on the GO TO statement.

1.4.3. Object-Oriented Programming.

- The idea behind OOP is that, a computer program is composed of a collection of *individual units, or objects* as opposed to traditional view in which a program is a list of instructions to the computer.
- Object-oriented programming is claimed to give more flexibility, easing changes to programs. The OOP approach is often simpler to develop and maintain.

1.5. Problem solving process and software engineering.

1.5.1. Software Engineering.

- Software engineering is the profession that creates and maintains software applications by applying technologies and practices from *computer science*, *project management*, *engineering*, *application domain* and other fields.
- The method used in solving problems in computer science and/or information systems is called the *software development life cycle*.
- The software development life cycle has the following components.
  - ***preliminary investigation***
  - ***analysis***
  - ***design***
  - ***implementation***
  - ***testing and maintenance***

1.5.2. Problem Solving

- Problem solving is the process of transforming the description of a problem into the solution by using our knowledge of the problem domain and by relying on our ability to select and use appropriate problem-solving strategies, techniques, and tools.
- A problem is an *undesirable situation* that prevents the organization from fully achieving its purpose, goals and objectives. Or problem can also be defined as the *gap between the existing and the desired situation* where problem solving will try to fill this gap.
- There are two approaches of problem solving:
  - *Top down design:* is a systematic approach based on the concept that the structure of the problem should determine the structure of the solution and what should be done in lower level. This approach will

try to disintegrate a larger problem into more smaller and manageable problems to narrow the problem domain.

- *Bottom up design:* is the reverse process where the lowest level component are built first and the system builds up from the bottom until the whole process is finally completed.

1.6. Basic Program development tips

- The program we design in any programming language need to be:
  - ***Reliable***: the program should always do what it is expected to do and handle all types of exception.
  - ***Maintainable***: the program should be in a way that it could be modified and upgraded when the need arises.
  - ***Portable***: It needs to be possible to adapt the software written for one type of computer to another with minimum modification.
  - ***Efficient***: the program should be designed to make optimal use of time, space and other resources of the computer.

1.7. Algorithm designing and modeling the logic (using flow chart).

- A digital computer is a useful tool for solving a great variety of **problems**. A solution to a problem is called an **algorith**m; it describes the sequence of steps to be performed for the problem to be solved.
- Generally, an algorithm is a finite set of well-defined instructions for accomplishing some task which, given an initial state, will terminate in a corresponding recognizable end-state.
- The algorithm should be:
  - ***Precise and unambiguous***
  - ***Simple***
  - ***Correct***
  - ***Efficient***

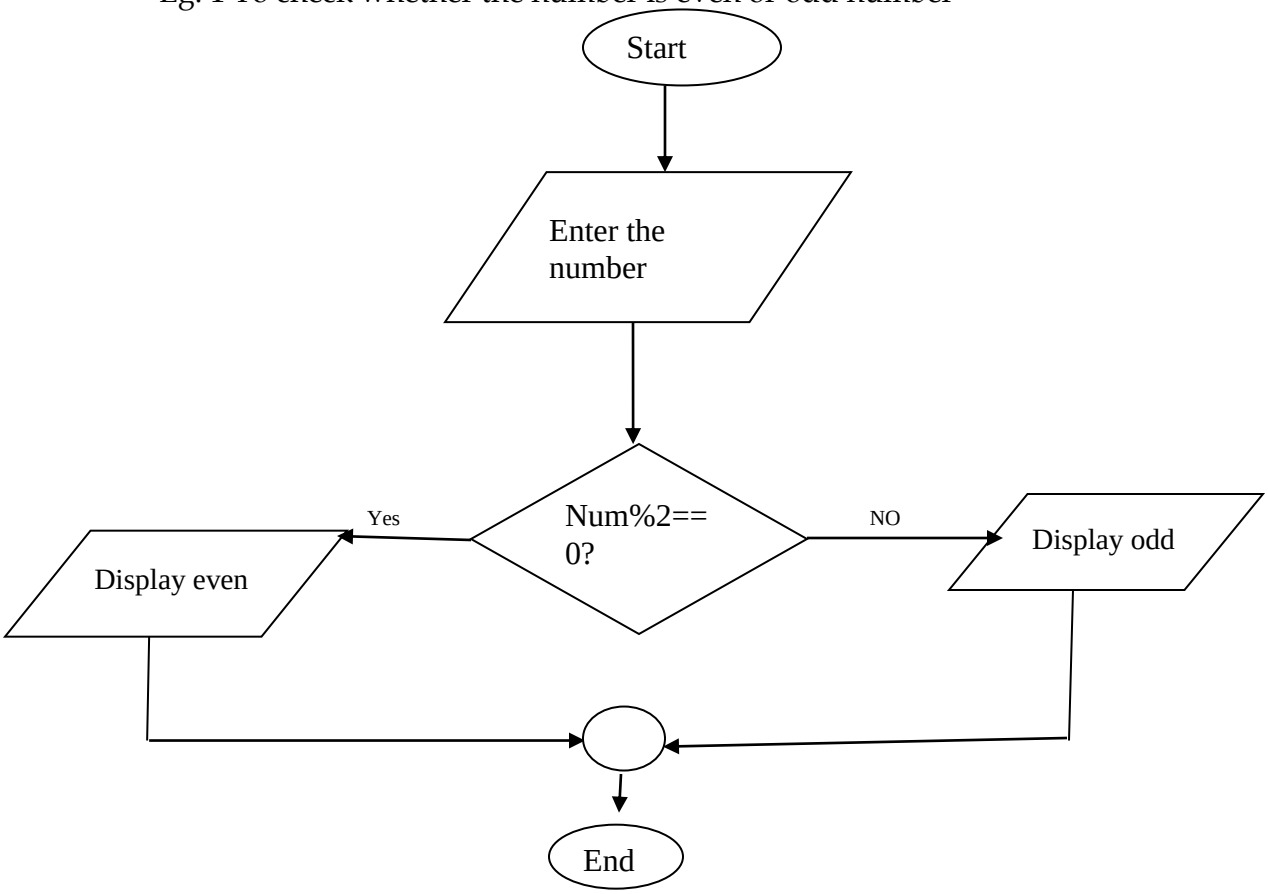
1.7.1. Modeling a programs logic using flow chart

- Algorithm could be designed using many techniques and tools. One tool of designing algorithm is by using flowcharts. Flowchart is a graphical way of expressing the steps needed to solve a problem.
- A flow chart is a schematic (*diagrammatic description*) representation of a process.

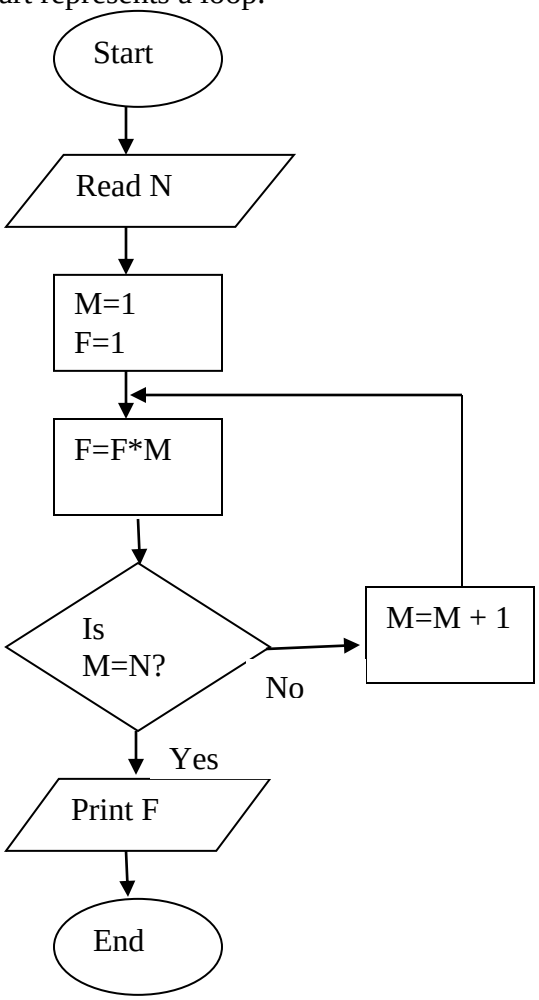
- Basic flowcharting symbols are:

	Terminal point
	Process
	Decision
	Input /output
	Flow line
	Inter-page connector
	On page connector

Eg. 1 To check whether the number is even or odd number



Example #2- A flowchart for computing factorial N (N!) where N! = (1 * 2 * 3 * ... * N). This flowchart represents a loop.



Pseudo Code- is a compact and informal high-level description of a computer programming algorithm that uses the structural conventions of a programming language, but is intended for human reading rather than machine reading. Pseudo-code typically omits details that are not essential for human understanding of the algorithm, such as variable declarations, system-specific code and subroutines.

Eg. 1. Pseudo code to check whether the number is even or odd number

```
Get number
If number% 2 = =0
    display even
Else
    display odd
halt
```

Eg. 2. Pseudo code for computing factorial N (N!) where N! = (1 * 2 * 3 * ... * N).

```
get number
factorial = 1
count = 1
while count < =number
        factorial = factorial *count
        count = count + 1
display factorial
halt
```

Compilers and Interpreters

Any program written in a language other than machine language needs to be translated to machine language. The set of instructions that do this task are known as translators. There are different kinds of translator software, among which compilers and interpreters are of interest for most programmers.

Compilers: a compiler is a computer program that translates a series of statements written in source code (a collection of statements in a specific programming language) into a resulting object code (translated instructions of the statements in a programming language). A compiler changes or translates the whole source code into executable machine code (also called object code) which is output to a file for latter execution. E.g. C++, Pascal, FORTRAN, etc.

Interpreters: is a computer program that translates a single high level statement and executes it and then goes to the next high level language line etc. E.g. QBASIC, Lisp etc.

Mechanics of Creating a program (C++)

C++ programs typically go through *five phases* to be executed: these are *edit*, *preprocess*, *compile*, *link*, *load*:

1. *Edit*: this is accomplished with an editor program. The programmer types C++ statements with the editor and makes corrections if necessary. The programs source file is then stored on *secondary storage device* such as a disk with a ".cpp" file name.

   After the program is edited, C+ principally compiled in three phases:

   preprocessing, ***translation to object code***, and ***linking*** (the last two phases are what is generally thought of as the "compilation" process).

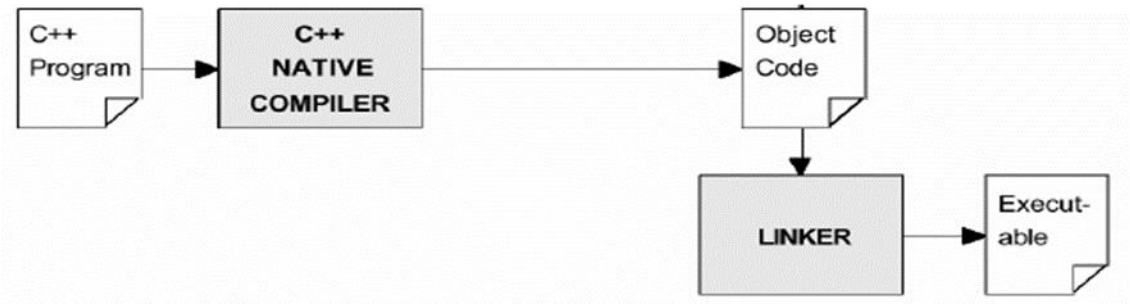
2. *Preprocess*: In a C++ system, a *preprocessor* program execute automatically before the compiler's translation phase begins. The C++ preprocessor obeys command called *preprocessor directives*, which indicate that certain manipulations are to be performed on the program before compilation. The preprocessor is invoked by the compiler before the program is converted to machine language. The C++ **preprocessor** goes over the program text and carries out the instructions specified by the preprocessor directives (e.g., #include). The result is a modified program text which no longer contains any directives.

3. *Compile*: Then, the C++ **compiler** translates the program code. The compiler may be a true C++ compiler which generates native (assembly or machine) code. The outcome may be incomplete due to the program referring to library routines which are not defined as a

part of the program. For example, the << operator which is actually defined in a separate IO library.

4. *Linking*: C++ program typically contain references to functions and data defined elsewhere, such as in the standard libraries. The object code produced by the C++ compiler typically contains "*holes*" due to these missing parts. A linker links the *object code* with the code for the missing function to produce an *executable image* (with no missing pieces). Generally, the **linker** completes the object code by linking it with the object code of any library modules that the program may have referred to. The final result is an executable file
5. *Loading*: the loader takes the executable file from disk and transfers it to memory. Additional components from shared libraries that support the program are also loaded. Finally, the computer, under the control of its CPU, executes the program.

   In practice all these steps are usually invoked by a single command and the user will not even see the intermediate files generated.



Worksheet 1

For each of the problems below, develop a flow chart

1) Receive a number and determine whether it is odd or even.

2) Obtain two numbers from the keyboard, and determine and display which (if either) is the larger of the two numbers.

3) Receive 3 numbers and display them in ascending order from smallest to largest

4) Add the numbers from 1 to 100 and display the sum

5) Add the even numbers between 0 and any positive integer number given by the user.

6) Find the average of two numbers given by the user.

7) Find the average, maximum, minimum, and sum of three numbers given by the user.

8) Find the area of a circle where the radius is provided by the user.

9) Swap the contents of two variables using a third variable.

10) Swap the content of two variables without using a third variable.

11) Read an integer value from the keyboard and display a message indicating if this number is odd or even.

12) Read 10 integers from the keyboard in the range 0 - 100, and count how many of them are larger than 50, and display this result.

13) Take an integer from the user and display the factorial of that number